# SHILATE

## Software Hardware In Loop Automotive Testing Environment

Tudor-Ciprian Tămaș

Supervisor: Sl. Dr. Ing. Valentin Mărănescu

*Electronics, Telecommunications and Information Technology*
Politehnica University of Timișoara

July 2026

**The Industry Shift**

- Vehicles are becoming software-defined — behaviour updated over-the-air
- Reinforcement learning (RL) is maturing as a practical control tool
- But no open-source pipeline bridges **RL training** and **SDV deployment**

**Four Open Challenges**

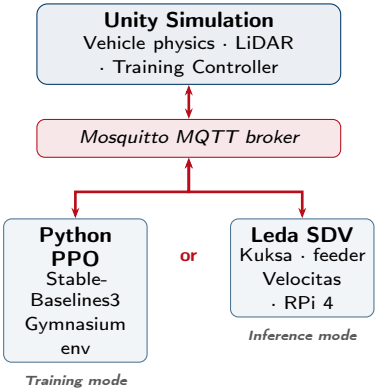
1. Simulation $\leftrightarrow$ middleware protocol coupling
2. Standardised signal representation (VSS)
3. Operational safety monitoring of learned policies
4. Reproducible, one-click training workflow

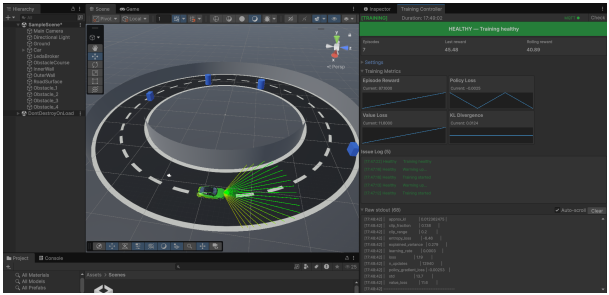
**Goal**

Build an **end-to-end, open-source** pipeline that:

- Trains a deep RL agent in a Unity physics sim
- Deploys the policy onto a real Eclipse Leda SDV stack
- Uses the **same MQTT / VSS layer** from training to hardware

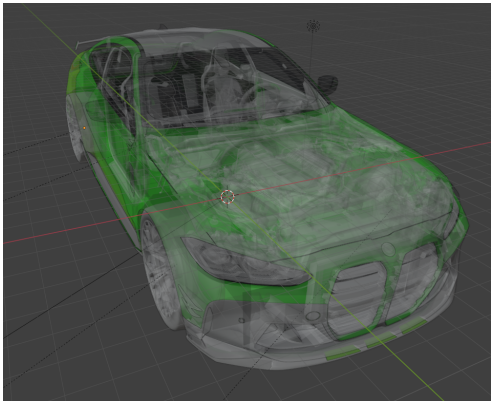
# System Architecture — Training vs. Inference Mode



Simulation environment overview

- **Training**: Unity Sim ↔ MQTT ↔ Python PPO
- **Deploy**: Unity Sim ↔ MQTT ↔ Eclipse Leda SDV
- Same `env0/` topic namespace in both modes — no protocol changes at deploy time



Rear-wheel-drive vehicle model

**Vehicle & Sensor**

- 1 500 kg RWD chassis — four WheelColliders, Pacejka tire model
- 21-ray LiDAR spanning 120° at 15 m range, published at 10 Hz
- Observation $o \in \mathbb{R}^{23}$: rays + normalised speed + steer

**Reward Function**

$$r_t = \underbrace{2\,\Delta p_t}_{\text{progress}} + \underbrace{(-5)\,1_{\text{collision}}}_{\text{collision}} + \underbrace{+50\,1_{\text{lap}} \;+\; +25\,1_{\frac{1}{2}\text{-turn}}}_{\text{milestones}}$$

**Training Controller (Editor window)**

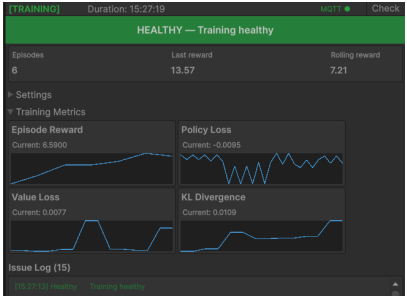
- One-click **Start Training / Run Model / Stop**
- Live health banner — 5 failure modes monitored
- Real-time reward, loss, and KL-divergence graphs

## Algorithm & Architecture

- Proximal Policy Optimisation (PPO)
- Actor-Critic MLP: two hidden layers [256, 128]
- Action space: $a = [\delta, \theta, \beta]$ — steer $\in [-1, 1]$, throttle $\in [0, 1]$, brake $\in [0, 1]$

## Key Hyperparameters

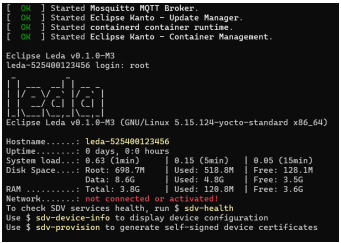
| Parameter | Value |
|---|---|
| Total timesteps | 100 000 |
| Learning rate | $3 \times 10^{-4}$ |
| Rollout steps | 2 048 |
| Batch size | 64 |
| Clip $\epsilon$ / $\gamma$ | 0.2 / 0.999 |




Training convergence graphs (TensorBoard)

- Gymnasium-compliant wrapper — any SB3 algorithm can be substituted
- Structured telemetry: `SHILATE-METRIC` / `SHILATE-HEALTH` markers parsed live by the Editor
- 5-second observation timeout triggers health alert

# Eclipse Leda SDV Integration




Eclipse Leda container stack

## 5-Container Docker Compose Stack

1. **Mosquitto** — MQTT broker
2. **Kuksa databroker** — VSS signal store
3. **mqtt-kuksa-feeder** — MQTT → VSS bridge
4. **Velocitas app** — safety monitor
5. **leda-controller** — RL inference engine

## Signal Chain

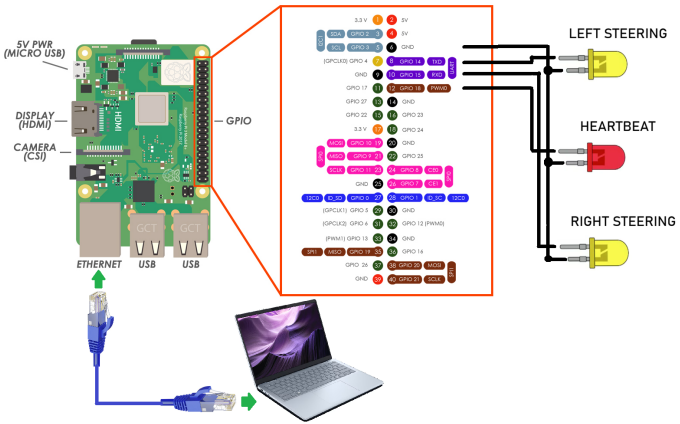
- `env0/vehicle/speed` → `Vehicle.Speed` (COVESA VSS)
- Velocitas app: overspeed alert (>120 km/h), RPM alert (>6 500)

## Deployment to Raspberry Pi 4

- `deploy-pi.sh` — cross-compiles ARM64 via `docker buildx`, transfers and starts stack
- Full deploy cycle: ≈14 minutes

**Remote Inference** — Training Controller targets the Pi broker; no code changes, policy runs on-device

# Hardware Setup — Raspberry Pi 4 Connections




## Physical Wiring

- **Ethernet** — direct cable to development laptop running Unity Sim and MQTT broker
- **3 GPIO LEDs** (via UART pins) used as hardware status indicators
- MQTT traffic flows over the LAN link using the same `env0/` topic namespace — zero reconfiguration
- 15–20 ms round-trip latency measured over direct Ethernet

**Key insight**

The same containerised inference stack runs identically on the Pi and inside the QEMU Leda image.

**Training Convergence** (3 seeds, mean)

| Milestone | Timestep | $\sigma$ |
|---|---|---|
| First collision-free episode | 12 400 | $\pm$2 100 |
| Half-turn bonus triggered | 31 000 | $\pm$3 800 |
| First full lap | 48 500 | $\pm$5 200 |
| Rolling mean $\geq 60$ | 76 000 | $\pm$6 900 |

**Inference — 100 episodes (local)**

| Metric | Value |
|---|---|
| Lap completion rate | 97% |
| Mean episode reward | 63.4 |
| Mean lap duration | 38.2 s |
| Mean speed (completed laps) | 47.6 km/h |
| Collisions per episode | 0.01 |

**Ablation Study**

- Half-turn bonus removed ($c_h = 0$): first lap delayed by $\approx$18 000 steps; rolling mean 12 pts lower at 100K
- Weaker collision penalty ($c_c = -1$): laps completed but agent drove closer to walls

**Health Monitoring — All Failure Modes Detected**

- Observation timeout ($> 5$ s) $\rightarrow$ *Critical*
- Python crash $\rightarrow$ non-zero exit code
- MQTT disconnect $\rightarrow$ *Disconnected* within 35 s

**Raspberry Pi Remote Inference**

| Metric | Value |
|---|---|
| Lap completion rate (LAN) | 95% |
| Additional latency (LAN) | 15–20 ms |
| Deploy time | $\approx$14 min |

## Conclusions

**What was achieved**

- End-to-end RL → SDV pipeline — same MQTT/VSS protocol from Unity training to Raspberry Pi 4 hardware
- Agent converges within 100 000 timesteps; 97% lap completion in inference
- One-click Training Controller with 5-failure-mode health monitoring
- Validated on physical hardware — 95% remote lap completion over LAN

**Future Work**

- Implementing a camera sensor and training a vision-based agent
- Safety monitor → active command override (not just alerts)
- Multi-agent scenarios and cooperative driving
- CAN bus hardware-in-loop testing on real ECUs

Universitatea Politehnica din Timișoara

**Thank you!** Questions?